Skin Lesion Classification with Explainability

Lenon, Danielle Laurence B.
School of Computing, Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario, Angeles City, Pampanga, Philippines
+63 932 647 6559
dblenon@student.hau.edu.ph

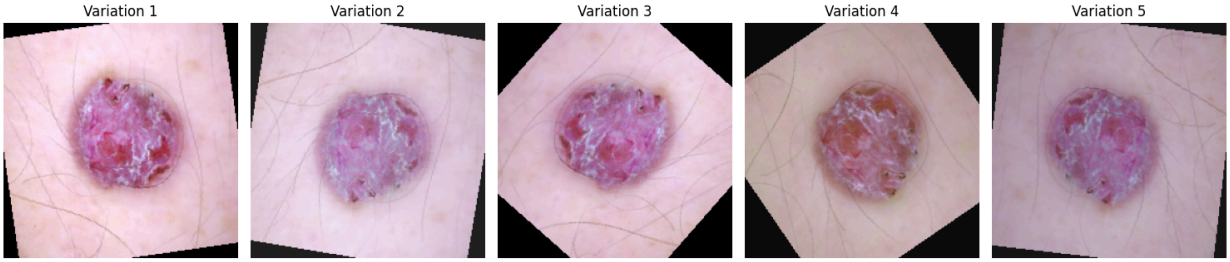
Liwag, Seiji Luis S.
School of Computing, Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario, Angeles City, Pampanga, Philippines
+63 961 061 0584
ssliwag1@student.hau.edu.ph

Ochoa, Bianca Claire L.
School of Computing, Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario, Angeles City, Pampanga, Philippines
+63 919 929 1848
blochoa@student.hau.edu.ph

Ochoa, Bianca Venice L.
School of Computing, Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario, Angeles City, Pampanga, Philippines
+63 919 921 7410
blochoa1@student.hau.edu.ph

Project and Dataset

The goal of the project is to establish a baseline CNN model and iteratively train toward a balanced model that is usable in classifying skin lesions. We implemented Transfer Learning (EfficientNet-B4, trained in ImageNet) and utilized HAM10000 as the primary dataset. Before training, the dataset was analyzed for its class distribution and it was identified that there was a significant imbalance between the malignant and benign class, with a ratio of approximately 80:20 difference. To counter the class imbalance, we utilized WeightRandomSampler, adjusted the loss function, and implemented data augmentation pipelines to compensate.



Sample of data augmentation (rotation, flip, and color jitter):

Initial Performance Metrics

In the model's first training (v1), the best performance was produced from Epoch 20 with an accuracy of 90.51% and F1 of 0.7689. This indicates that the model *could* successfully classify the majority of the skin lesions. However, the model produced a Recall of 0.6718 and Precision of 0.8344. This indicates that there is an approximately 33% chance that malignant skin lesions would be misclassified as benign, which introduces a massive risk in overall security due to misdiagnosis. The HAM10000 dataset is known for being imbalanced due to 80% of its images being benign. The model likely achieved high accuracy by becoming biased and classifying the majority of the validation images as benign, and only ever flagged malignant if it is 100% certain and confident. Due to the class imbalance of the dataset, further improvements and training will focus on Precision, Recall, and F-1 Score in its evaluation metrics while looking for a balance with accuracy. The training used optim.Adam, which has a tendency to let internal network weights to grow unconstrained, and it essentially results in the model memorizing the pixels of the training images rather than learning the malignant features and patterns. It also kept a learning rate of 0.0001 throughout, and there was no decay scheduler to keep watch for the validation loss.

Epoch [20/30] - Train Loss: 0.1164 | Val Loss: 0.2705 | Val Acc: 0.9051 | Val F1: 0.7689 (Recall and Precision was obtained manually after training)

```
Epoch [1/30] - Train Loss: 0.6737 | Val Loss: 0.5623 | Val Acc: 0.7423 | Val F1: 0.5521
Epoch [2/30] - Train Loss: 0.4903 | Val Loss: 0.4418 | Val Acc: 0.7932 | Val F1: 0.5981
Epoch [3/30] - Train Loss: 0.4323 | Val Loss: 0.4077 | Val Acc: 0.7882 | Val F1: 0.5891
Epoch [4/30] - Train Loss: 0.3928 | Val Loss: 0.3837 | Val Acc: 0.8142 | Val F1: 0.6367
Epoch [5/30] - Train Loss: 0.3639 | Val Loss: 0.3524 | Val Acc: 0.8302 | Val F1: 0.6559
Epoch [6/30] - Train Loss: 0.3444 | Val Loss: 0.3962 | Val Acc: 0.8112 | Val F1: 0.6494
Epoch [7/30] - Train Loss: 0.3201 | Val Loss: 0.3132 | Val Acc: 0.8611 | Val F1: 0.7011
Epoch [8/30] - Train Loss: 0.2974 | Val Loss: 0.2843 | Val Acc: 0.8711 | Val F1: 0.7007
Epoch [9/30] - Train Loss: 0.2806 | Val Loss: 0.2861 | Val Acc: 0.8741 | Val F1: 0.7097
Epoch [10/30] - Train Loss: 0.2560 | Val Loss: 0.3007 | Val Acc: 0.8741 | Val F1: 0.7249
Epoch [11/30] - Train Loss: 0.2409 | Val Loss: 0.2957 | Val Acc: 0.8711 | Val F1: 0.7165
Epoch [12/30] - Train Loss: 0.2060 | Val Loss: 0.2914 | Val Acc: 0.8731 | Val F1: 0.7159
Epoch [13/30] - Train Loss: 0.2031 | Val Loss: 0.2590 | Val Acc: 0.8901 | Val F1: 0.7406
Epoch [14/30] - Train Loss: 0.1739 | Val Loss: 0.2819 | Val Acc: 0.8961 | Val F1: 0.7374
Epoch [15/30] - Train Loss: 0.1583 | Val Loss: 0.2691 | Val Acc: 0.9001 | Val F1: 0.7561
Epoch [16/30] - Train Loss: 0.1505 | Val Loss: 0.2892 | Val Acc: 0.8951 | Val F1: 0.7445
Epoch [17/30] - Train Loss: 0.1282 | Val Loss: 0.2720 | Val Acc: 0.9061 | Val F1: 0.7662
Epoch [18/30] - Train Loss: 0.1291 | Val Loss: 0.3171 | Val Acc: 0.8881 | Val F1: 0.7346
Epoch [19/30] - Train Loss: 0.1153 | Val Loss: 0.3234 | Val Acc: 0.8841 | Val F1: 0.7041
Epoch [20/30] - Train Loss: 0.1164 | Val Loss: 0.2705 | Val Acc: 0.9051 | Val F1: 0.7689
Epoch [21/30] - Train Loss: 0.0979 | Val Loss: 0.3308 | Val Acc: 0.8881 | Val F1: 0.7407
Epoch [22/30] - Train Loss: 0.0919 | Val Loss: 0.3172 | Val Acc: 0.8971 | Val F1: 0.7406
Epoch [23/30] - Train Loss: 0.0881 | Val Loss: 0.3348 | Val Acc: 0.8911 | Val F1: 0.7423
Epoch [24/30] - Train Loss: 0.0774 | Val Loss: 0.3490 | Val Acc: 0.8941 | Val F1: 0.7282
Epoch [25/30] - Train Loss: 0.0755 | Val Loss: 0.3323 | Val Acc: 0.9101 | Val F1: 0.7443
Epoch [26/30] - Train Loss: 0.0688 | Val Loss: 0.3042 | Val Acc: 0.9031 | Val F1: 0.7557
Epoch [27/30] - Train Loss: 0.0612 | Val Loss: 0.3193 | Val Acc: 0.9051 | Val F1: 0.7570
Epoch [28/30] - Train Loss: 0.0504 | Val Loss: 0.3726 | Val Acc: 0.9011 | Val F1: 0.7506
Epoch [29/30] - Train Loss: 0.0505 | Val Loss: 0.3477 | Val Acc: 0.9001 | Val F1: 0.7396
Epoch [30/30] - Train Loss: 0.0479 | Val Loss: 0.3115 | Val Acc: 0.9061 | Val F1: 0.7513
Training Complete. Models saved.
```

In the model's second training (v2), we changed the weight from class_sample_count (v1) to np.sqrt(class_sample_count). It was advised in forums that wrapping it in a square root would lessen the penalty, which results in the model prioritizing malignant lesions while also classifying benign in equal confidence, essentially providing better balance between the two. We also changed the learning rate from 0.0001 to 0.0002, and included the displaying of Recall and Precision during each epoch since the former version only included train and validation loss, validation F1 and accuracy. This training (v2) produced the best performance from Epoch 18 (before Google Colab cut off due to free tier limitations). It achieved an accuracy of 90.31%, Recall of 0.8410, and Precision of 0.7130, which is considerably more balanced than previous metrics (v1). However, for further improvement, we will rework the program to ensure that instead of saving the weight of every 5 epochs and the final epoch, we will save the epoch with the best metrics.

Epoch [18/30] - Train Loss: 0.0584 | Val Loss: 0.3438 | Val Acc: 0.9031 | Val F1: 0.7718 | Recall: 0.8410 | Prec: 0.7130

```
Starting Training for 30 Epochs...
Epoch [1/30] - Train Loss: 0.5629 | Val Loss: 0.5915 | Val Acc: 0.7163 | Val F1: 0.5463 | Recall: 0.8769 | Prec: 0.3968
Epoch [2/30] - Train Loss: 0.3995 | Val Loss: 0.4747 | Val Acc: 0.7722 | Val F1: 0.5972 | Recall: 0.8667 | Prec: 0.4555
Epoch [3/30] - Train Loss: 0.3476 | Val Loss: 0.5942 | Val Acc: 0.7403 | Val F1: 0.5899 | Recall: 0.9590 | Prec: 0.4260
Epoch [4/30] - Train Loss: 0.3126 | Val Loss: 0.3772 | Val Acc: 0.8282 | Val F1: 0.6504 | Recall: 0.8205 | Prec: 0.5387
Epoch [5/30] - Train Loss: 0.2935 | Val Loss: 0.4493 | Val Acc: 0.7962 | Val F1: 0.6383 | Recall: 0.9231 | Prec: 0.4878
Epoch [6/30] - Train Loss: 0.2671 | Val Loss: 0.4611 | Val Acc: 0.7982 | Val F1: 0.6431 | Recall: 0.9333 | Prec: 0.4906
Epoch [7/30] - Train Loss: 0.2272 | Val Loss: 0.3077 | Val Acc: 0.8691 | Val F1: 0.7183 | Recall: 0.8564 | Prec: 0.6185
Epoch [8/30] - Train Loss: 0.2106 | Val Loss: 0.3274 | Val Acc: 0.8711 | Val F1: 0.7177 | Recall: 0.8410 | Prec: 0.6260
Epoch [9/30] - Train Loss: 0.1834 | Val Loss: 0.3082 | Val Acc: 0.8921 | Val F1: 0.7391 | Recall: 0.7846 | Prec: 0.6986
Epoch [10/30] - Train Loss: 0.1680 | Val Loss: 0.3113 | Val Acc: 0.8901 | Val F1: 0.7454 | Recall: 0.8256 | Prec: 0.6793
Epoch [11/30] - Train Loss: 0.1573 | Val Loss: 0.2967 | Val Acc: 0.8861 | Val F1: 0.7273 | Recall: 0.7795 | Prec: 0.6816
Epoch [12/30] - Train Loss: 0.1243 | Val Loss: 0.3332 | Val Acc: 0.8861 | Val F1: 0.7373 | Recall: 0.8205 | Prec: 0.6695
Epoch [13/30] - Train Loss: 0.1217 | Val Loss: 0.3163 | Val Acc: 0.8901 | Val F1: 0.7393 | Recall: 0.8000 | Prec: 0.6872
Epoch [14/30] - Train Loss: 0.1055 | Val Loss: 0.3575 | Val Acc: 0.8931 | Val F1: 0.7318 | Recall: 0.7487 | Prec: 0.7157
Epoch [15/30] - Train Loss: 0.1037 | Val Loss: 0.3778 | Val Acc: 0.8841 | Val F1: 0.7327 | Recall: 0.8154 | Prec: 0.6653
Epoch [16/30] - Train Loss: 0.0773 | Val Loss: 0.3472 | Val Acc: 0.8941 | Val F1: 0.7464 | Recall: 0.8000 | Prec: 0.6996
Epoch [17/30] - Train Loss: 0.0672 | Val Loss: 0.3583 | Val Acc: 0.9041 | Val F1: 0.7681 | Recall: 0.8154 | Prec: 0.7260
Epoch [18/30] - Train Loss: 0.0584 | Val Loss: 0.3438 | Val Acc: 0.9031 | Val F1: 0.7718 | Recall: 0.8410 | Prec: 0.7130
Epoch [19/30] - Train Loss: 0.0585 | Val Loss: 0.3339 | Val Acc: 0.8981 | Val F1: 0.7548 | Recall: 0.8051 | Prec: 0.7104
Epoch [20/30] - Train Loss: 0.0578 | Val Loss: 0.3446 | Val Acc: 0.8981 | Val F1: 0.7524 | Recall: 0.7949 | Prec: 0.7143
Epoch [21/30] - Train Loss: 0.0517 | Val Loss: 0.3589 | Val Acc: 0.8931 | Val F1: 0.7409 | Recall: 0.7846 | Prec: 0.7018
```

train.py

```python
#if you are getting errors, run the gitclone (first code block) and only run this code block
#efficientnet-v2 (AdamW + Square Root Class Weights + Recall Metrics)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, WeightedRandomSampler
from torchvision import datasets, transforms
import timm
import numpy as np
import sys
import os
from sklearn.metrics import f1_score, accuracy_score, recall_score, precision_score

#directory validation
sys.path.append(os.path.abspath(os.path.join(os.getcwd(), 'source')))
from source.augment_data import train_transforms, val_transforms

#device configuration for better performance
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

#folders and applying data augmentation pipelines
train_dir = '/content/skin-lesion-classifier/data/model_data/train'
val_dir = '/content/skin-lesion-classifier/data/model_data/val'
train_dataset = datasets.ImageFolder(train_dir, transform=train_transforms)
val_dataset = datasets.ImageFolder(val_dir, transform=val_transforms)

#weightedrandomsampler for the 80/20 imbalance (balances the batches)
target = np.array(train_dataset.targets)
class_sample_count = np.array([len(np.where(target == t)[0]) for t in np.unique(target)])
sampler_weights = 1. / class_sample_count
samples_weight = np.array([sampler_weights[t] for t in target])
sampler = WeightedRandomSampler(torch.from_numpy(samples_weight), len(samples_weight))

train_loader = DataLoader(train_dataset, batch_size=16, sampler=sampler, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False, num_workers=2, pin_memory=True)

#efficientNet-B4
model = timm.create_model('efficientnet_b4', pretrained=True, num_classes=2)
model = model.to(device)

#loss and optimizer
loss_weights = 1. / np.sqrt(class_sample_count)
weights_tensor = torch.FloatTensor(loss_weights).to(device)
criterion = nn.CrossEntropyLoss(weight=weights_tensor)

#adamW - Learning rate slightly increased to help break out of local minimums
optimizer = optim.AdamW(model.parameters(), lr=0.0002, weight_decay=0.01)

#learning rate scheduler to reduce LR when Val Loss plateaus
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

#training loop limited to 30 epochs
num_epochs = 30
print(f"Starting Training for {num_epochs} Epochs...")

for epoch in range(num_epochs):
    #training
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    avg_train_loss = running_loss / len(train_loader)
```

```python
    #validation
    model.eval()
    all_preds = []
    all_labels = []
    val_loss = 0.0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    # evaluation metrics
    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds)
    recall = recall_score(all_labels, all_preds)
    precision = precision_score(all_labels, all_preds)

    avg_val_loss = val_loss / len(val_loader)
    scheduler.step(avg_val_loss)

    print(f"Epoch [{epoch+1}/{num_epochs}] - Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f} | Val Acc: {acc:.4f} | Val F1: {f1:.4f} | Recall: {recall:.4f} | Prec: {precision:.4f}")

    #save the best performing model
    if (epoch + 1) % 5 == 0:
        torch.save(model.state_dict(), f'efficientnet_b4_v2_epoch_{epoch+1}.pth')

#save the final model
torch.save(model.state_dict(), 'efficientnet_b4_v2_baseline_final.pth')
print("Training Complete. Models saved.")
```

User Testing

After training, a GUI user testing script (user_test.py) was developed to allow users to test user-input images of skin lesions for initial performance evaluation. The script accepts an image path as input and returns the classification label (Benign/Malignant) and the confidence scores for each classification. To improve prediction stability, as initial testing proved that standard user-input with normalization often resulted in unexpected results or misclassification, Test Time Augmentation (TTA) was implemented, running 8 reformulations of the user-input image which includes:

- Original
- Horizontal flip
- Vertical flip
- 45°, 90°, 135°, and 180° degree rotations, and
- Color jitters

After image reformulation, the probability values were calculated and averaged before producing a final result. To determine whether the image reformulation is effective in improving classification, both standard (no augmentation or image reformulation) and TTA results are displayed side by side for comparison.

Testing

Benign-Test.png

```
Model loaded from '/content/skin-lesion-classifier/models/efficientnetb4-v2/efficientnet_b4_v3_epoch_15.pth' on cuda
Enter image path (or 'q' to quit): /content/skin-lesion-classifier/user-testing-own/Benign-Test.png

File : Benign-Test.png

[Without Image Reformulation]
Prediction  : Benign

Confidence Scores:
Benign      : 100.00%
Malignant   : 0.00%

[With TTA (8 Reformulation Averaged)]
Prediction  : Benign

Confidence Scores:
Benign      : 100.00%
Malignant   : 0.00%

NOTE: For research use and testing only.

Enter image path (or 'q' to quit):
```

Benign-Test2.png

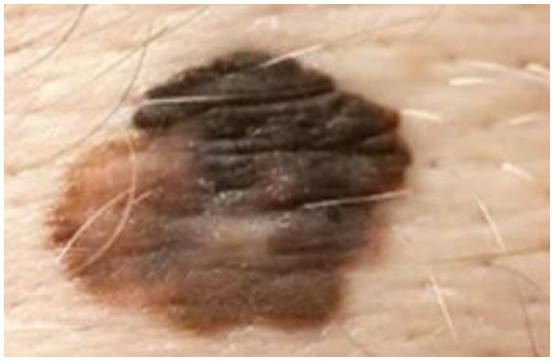
```
Enter image path (or 'q' to quit): /content/skin-lesion-classifier/user-testing-own/Benign-Test2.png

────────────────────────────────────────
File : Benign-Test2.png
────────────────────────────────────────
[Without Image Reformulation]
Prediction  : Benign

Confidence Scores:
Benign      : 87.05%
Malignant   : 12.95%
────────────────────────────────────────
[With TTA (8 Reformulation Averaged)]
Prediction  : Benign

Confidence Scores:
Benign      : 76.18%
Malignant   : 23.82%
────────────────────────────────────────
NOTE: For research use and testing only.
────────────────────────────────────────
```

Melanoma-Malignant.png



```
Enter image path (or 'q' to quit): /content/skin-lesion-classifier/user-testing-own/Melanoma-Malignant.jpg

────────────────────────────────────────
File : Melanoma-Malignant.jpg
────────────────────────────────────────
[Without Image Reformulation]
Prediction  : Malignant

Confidence Scores:
Benign      : 9.21%
Malignant   : 90.79%
────────────────────────────────────────
[With TTA (8 Reformulation Averaged)]
Prediction  : Malignant

Confidence Scores:
Benign      : 20.67%
Malignant   : 79.33%
────────────────────────────────────────
NOTE: For research use and testing only.
────────────────────────────────────────
```

Melanoma-MalignantTest2.png

```
Enter image path (or 'q' to quit): /content/skin-lesion-classifier/user-testing-own/Melanoma-MalignantTest2.png

────────────────────────────────────────
File : Melanoma-MalignantTest2.png
────────────────────────────────────────
[Without Image Reformulation]
Prediction  : Benign

Confidence Scores:
Benign      : 54.55%
Malignant   : 45.45%
────────────────────────────────────────
[With TTA (8 Reformulation Averaged)]
Prediction  : Malignant

Confidence Scores:
Benign      : 16.22%
Malignant   : 83.78%
────────────────────────────────────────
NOTE: For research use and testing only.
────────────────────────────────────────
```

user_test.py

```python
# with image reformulation for user testing, getting mean of reformulated prediction values

# gui user testing (not test images)
# confidence level, benign or malignant classification

import torch
import timm
import numpy as np
from PIL import Image
from torchvision import transforms
import os

# load model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

#haba wow
model_path = '/content/skin-lesion-classifier/models/efficientnetb4-v2/efficientnet_b4_v3_epoch_15.pth'
model = timm.create_model('efficientnet_b4', pretrained=False, num_classes=2)
model.load_state_dict(torch.load(model_path, map_location=device))
model = model.to(device)
model.eval()
print(f"Model loaded from '{model_path}' on {device}")

# image uploaded will be transformed for appropriate size
base_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# image reformulation
tta_transforms = [ base_transforms,
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomHorizontalFlip(p=1.0), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomVerticalFlip(p=1.0), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomRotation(degrees=(45, 45)), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomRotation(degrees=(90, 90)), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomRotation(degrees=(135, 135)), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.RandomRotation(degrees=(180, 180)), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
    transforms.Compose([transforms.Resize((224, 224)), transforms.ColorJitter(brightness=0.2, contrast=0.2), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])]),
]

# classify non-reformulated
def classify_standard(img):
    tensor = base_transforms(img).unsqueeze(0).to(device)
    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1).squeeze().cpu().numpy()
    return probs

# classify reformulated data
def classify_tta(img):
    all_probs = []
    with torch.no_grad():
        for t in tta_transforms:
            tensor = t(img).unsqueeze(0).to(device)
            probs  = torch.softmax(model(tensor), dim=1).squeeze().cpu().numpy()
            all_probs.append(probs)
    return np.mean(all_probs, axis=0)

def print_result(label_header, probs):
    pred_idx    = int(np.argmax(probs))
    label       = ["Benign", "Malignant"][pred_idx] # 0 = benign, 1 = malignant

    print(f"[{label_header}]")
    print(f"Prediction  : {label}")
    print("\nConfidence Scores: ")
    print(f"Benign      : {probs[0]*100:.2f}%")
    print(f"Malignant   : {probs[1]*100:.2f}%")
```

```python

# display classification results ito
def classify(image_path):
    if not os.path.isfile(image_path):
        print(f"[ERROR] File not found: {image_path}")
        return

    img = Image.open(image_path)
    img = img.convert("RGB")

    standard_probs = classify_standard(img)
    tta_probs      = classify_tta(img)

    print("\n" + "⊟" * 40)
    print(f"File : {os.path.basename(image_path)}")
    print("⊟" * 40)
    print_result("Without Image Reformulation", standard_probs)
    print("⊟" * 40)
    print_result(f"With TTA ({len(tta_transforms)} Reformulation Averaged)", tta_probs)
    print("⊟" * 40)
    print("NOTE: For research use and testing only.")
    print("⊟" * 40 + "\n")

# main na
while True:
    path = input("Enter image path (or 'q' to quit): ").strip()
    if path.lower() in ("q", "quit", "exit"):
        print("Exiting.")
        break
    classify(path)
```

Ethics Statement

The model will be trained using the HAM10000 dataset, which predominantly features lighter tones largely gathered from Austria and Australia. Performance dips occur when analyzing lesions found on less-represented skin varieties, prompting questions around equity and broader usability. Alongside that, case distribution may skew heavily, where roughly four out of five are non-cancerous, leaving just one-fifth labeled as cancerous. Here, exactness/accuracy may give a distorted view. Should every instance be labeled benign, the system reaches roughly 80% correctness, yet

overlooks harmful abnormalities. For this reason, attention shifts toward sensitivity, exactness, and the combined measure known as F1, placing weight on sensitivity to limit missed detections. Exactness matters too, since incorrect alarms may lead to undue concern.

A serious ethical risk emerges if individuals interpret results as medical verdicts, rather than seeking advice from healthcare professionals or conducting further research. It may happen that harmful growths get labeled harmless, particularly when false alarms are missed. To mitigate this risk, the system will display disclaimers that state that the model is purely for academic or research purposes, and is not for operational clinical use, nor will it replace actual consultations with healthcare professionals. Visuals highlight image areas affecting decisions. Grad-CAM and attention patterns will do this, while uncertainty levels come from repeated sampling trials. False negatives or inaccurate classifications pose a massive safety risk among users, especially regarding potentially cancerous skin lesions. To mitigate this, the model will integrate Grad-CAM and Saliency to provide visualization for the results to allow users to verify and interpret them themselves. Additionally, MC dropout will also provide confidence estimates, so that users will be made aware of low-confidence results and proceed with caution.